

UNIVERSIDAD DE SALAMANCA

GRADO EN INGENIERIA INFORMATICA

ANEXO VII

Manual de Montaje

Sistema de Gestion Documental mediante Blockchain



David Pérez Velasco

Tutores:

Gabriel Villarrubia González

Sergio García González

Julio 2026

Trabajo de Fin de Grado

Índice

1. Requisitos hardware y software	4
1.1. Requisitos de hardware	4
1.2. Software necesario	4
1.3. Requisitos de red	5
1.4. Sistema operativo	6
2. Arquitectura de despliegue	6
2.1. Visión general de contenedores	6
2.2. Descripción de servicios	7
2.3. Volúmenes persistentes	7
2.4. Red interna Docker	8
2.5. Flujo de peticiones	8
3. Instalacion paso a paso	9
3.1. Paso 1: Clonar el repositorio	9
3.2. Paso 2: Configurar variables de entorno	9
3.3. Paso 3: Levantar servicios con Docker Compose	10
3.4. Paso 4: Configurar IPFS	11
3.4.1. Alternativa: proveedor gestionado IPFS (Pinata)	11
3.4.2. Configuración del nodo propio como nodo de red	12
3.5. Paso 5: Desplegar smart contracts	12
3.5.1. Despliegue en red local	13
3.5.2. Despliegue en una red de pruebas testnet	13
3.6. Paso 6: Seed de base de datos	13
3.7. Paso 7: Configurar Nginx	13
3.8. Verificación post-instalación	14
4. Configuración completa del correo electrónico	15
4.1. Diagrama de flujo de envío de correos	15
4.2. Proceso interno de envío	15
4.3. Configuración SMTP en el backend	15
4.4. Configuración de Nodemailer	16
4.5. Variables relevantes de correo	16
4.6. Tipos de correo y criterios de envío	17
4.7. Resolución de problemas de entrega	17
5. Configuración de blockchain	17
5.1. Despliegue en red local (Hardhat network)	17
5.2. Despliegue en una red de pruebas testnet	18
5.3. Configuración de variables CONTRACT_ADDRESS y ADMIN_ROLE	18
5.4. Verificación del contrato en Etherscan	18
5.5. Configuración del event listener	19
6. Uso de Docker y Nginx	19
6.1. Comandos Docker de uso frecuente	20
6.2. Escalado y healthchecks en Docker Compose	20
6.3. Configuración avanzada de Nginx	20
6.4. Certificados SSL auto-renovables	21

7. GitHub Actions y despliegue continuo	21
7.1. Workflow completo de CI/CD	21
7.2. Descripción del pipeline	22
7.3. Configuración de secrets en GitHub	22
7.4. Estrategia de rollback	23
8. Mantenimiento y actualización	23
8.1. Backup de base de datos	23
8.2. Backup de IPFS	24
8.3. Actualización segura del sistema	24
8.4. Actualización de dependencias	24
8.5. Monitoreo de logs	24
8.6. Limpieza de recursos	25
9. Scripts de utilidad	25
9.1. Script de backup automatizado	25
9.2. Script de verificación de salud	25
9.3. Script de migración de datos	26
9.4. Script de generación de seeds	26
9.5. Reseed de entorno de desarrollo	26
10. Resolución de incidencias comunes	26
11. Referencias y bibliografía	28

Este anexo describe el procedimiento completo para poner en marcha DocumentChain, tanto en un entorno de producción como en uno de desarrollo local. Se ha buscado que el despliegue sea reproducible en cualquier servidor que disponga de Docker, sin necesidad de configuraciones manuales complejas.

A lo largo del documento se cubren los requisitos de hardware y software, la arquitectura de contenedores Docker y la configuración de los servicios externos: correo electrónico, blockchain e IPFS. También se detallan los pasos para levantar el sistema, la integración con Nginx como proxy inverso, el pipeline de despliegue continuo con GitHub Actions y la resolución de las incidencias más habituales que el autor se ha encontrado durante las pruebas.

1. Requisitos hardware y software

1.1. Requisitos de hardware

El despliegue de *DocumentChain* requiere un servidor capaz de ejecutar simultáneamente multiples contenedores Docker, un nodo IPFS y, en entornos de desarrollo, una red blockchain local. A continuación se detallan los requisitos mínimos y recomendados.

Componente	Mínimo	Recomendado	Observaciones
CPU	4 núcleos (x86_64)	8 núcleos o superior	Arquitectura AMD64; conjunto de instrucciones AVX2 recomendado para ciertas operaciones criptográficas
RAM	8 GB	16–32 GB	El nodo IPFS y la compilación de contratos inteligentes consumen memoria significativa en picos
Almacenamiento	50 GB SSD	200 GB NVMe SSD	IPFS almacena bloques de forma persistente; se recomienda monitorizar el crecimiento del volumen
Red	100 Mbps simétrico	1 Gbps simétrico	Conexión estable con latencia baja hacia Internet; ancho de banda de salida importante para envío de correos electrónicos

Cuadro 1: Requisitos hardware para el despliegue de DocumentChain

1.2. Software necesario

Todos los componentes de la aplicación se ejecutan dentro de contenedores Docker. No obstante, el servidor anfitrión debe disponer de las herramientas base para la gestión de imágenes, la clonación del código fuente y la ejecución de scripts de utilidad.

Software	Versión mín.	Versión rec.	Rol	Notas
Docker Engine	24.0.0	26.x	Motor de contenedores	Requiere cgroup v2 en kernels modernos
Docker Compose	2.20.0	2.27+	Orquestación de servicios	Plugin integrado en Docker Desktop; paquete separado en Linux
Node.js	20.0.0	20 LTS / 22 LTS	Runtime de scripts	Necesario para compilar contratos, ejecutar Prisma CLI y scripts de despliegue
npm	10.0.0	10+	Gestor de paquetes	Incluido con Node.js
Git	2.40	2.45+	Control de versiones	Para clonar y actualizar el repositorio
OpenSSL	3.0	3.2+	Generación de claves	Utilizado para crear JWT_SECRET y certificados autofirmados temporales
jq	1.6	1.7+	Procesamiento JSON	Opcional pero recomendado para automatización de healthchecks
certbot	2.0	2.10+	Certificados SSL	Requerido para Let's Encrypt en producción

Cuadro 2: Software base requerido en el servidor anfitrión

1.3. Requisitos de red

La infraestructura expone una serie de puertos que deben configurarse en el cortafuegos del servidor y, en su caso, en el panel del proveedor de nube.

Puerto	Protocolo	Servicio	Alcance	Descripción
80	TCP	nginx	0.0.0.0/0	Redirección HTTP a HTTPS
443	TCP	nginx	0.0.0.0/0	HTTPS: frontend y API REST protegidos
3000	TCP	backend	Red interna / 127.0.0.1	API REST; no debe exponerse públicamente sin proxy inverso
5001	TCP	ipfs	127.0.0.1	API administrativa de IPFS; restringir al anfitrión
8080	TCP	ipfs	0.0.0.0/0 (opc.)	Gateway IPFS pública; puede deshabilitarse si no se requiere
8545	TCP	hardhat	127.0.0.1	RPC JSON local de Hardhat; uso exclusivo en desarrollo
5432	TCP	postgres	Red interna	PostgreSQL; bloquear en el cortafuegos externo
25, 587	TCP	brevo-smtp	0.0.0.0/0 (saliente)	SMTP para envío de correos; el puerto 25 suele estar bloqueado en nubes públicas, preferir 587

Cuadro 3: Puertos de red utilizados por la plataforma

1.4. Sistema operativo

El entorno de producción recomendado es una distribución Linux de tipo Debian o Red Hat, preferentemente con soporte a largo plazo (LTS).

- **Ubuntu Server 22.04 LTS / 24.04 LTS:** probado y documentado en el presente manual.
- **Debian 12 (Bookworm):** alternativa estable con ciclo de parches extendido.
- **Windows 10/11:** válido únicamente para desarrollo local mediante Windows Subsystem for Linux 2 (WSL2) con una distribución Ubuntu.
- **macOS:** válido para desarrollo local; Docker Desktop para Mac gestiona la virtualización subyacente.

En producción se recomienda desactivar el inicio de sesión por contraseña para el usuario `root`, configurar autenticación por clave SSH y mantener el sistema actualizado mediante gestor de paquetes.

2. Arquitectura de despliegue

2.1. Visión general de contenedores

La aplicación se materializa como un conjunto de servicios orquestados mediante Docker Compose. La figura 1 ilustra la topología lógica de la infraestructura.

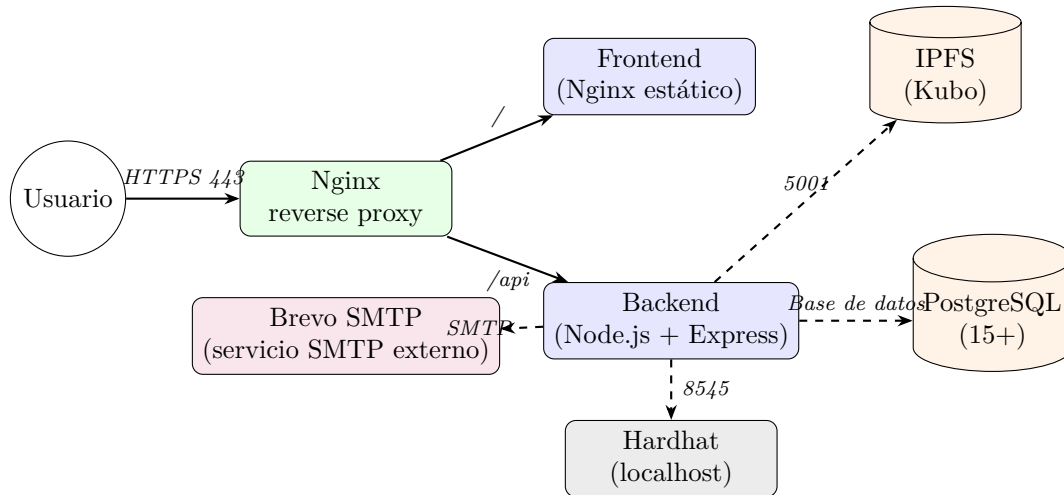


Figura 1: Diagrama de arquitectura de contenedores DocumentChain

2.2. Descripción de servicios

A continuación se describe la función de cada servicio definido en el archivo `docker-compose.yml`.

- **frontend**: construido a partir de una imagen base Nginx Alpine. Su responsabilidad consiste en servir los artefactos estáticos generados por Vite (HTML, JS, CSS) y resolver el enrutamiento de la Single Page Application (SPA) mediante la directiva `try_files`.
- **backend**: imagen basada en Node.js 20-slim. Ejecuta el servidor Express que expone la API REST en el puerto 3000. Contiene la lógica de negocio, la capa de cifrado de documentos y los adaptadores hacia base de datos, IPFS y blockchain.
- **postgres**: instancia de PostgreSQL 15 (o superior). Almacena usuarios, documentos meta-datos, versiones, firmas, notificaciones y logs de auditoría. Persiste sus datos en un volumen Docker dedicado.
- **ipfs**: nodo Kubo oficial. Gestiona el almacenamiento descentralizado de documentos cifrados. Expone la API en el puerto 5001 y opcionalmente un gateway de lectura en el 8080. Requiere configuración de CORS para la comunicación con el backend.
- **nginx**: proxy inverso frontal. Recibe el tráfico HTTPS en el puerto 443 y lo distribuye hacia el frontend o el backend según la ruta. Gestiona la terminación SSL, la compresión gzip y las cabeceras de seguridad HTTP.
- **brevo-smtp** (opcional pero recomendado): servicio SMTP externo de Brevo utilizado para el envío de correos electrónicos de la plataforma.
- **hardhat** (desarrollo local): nodo Ethereum local con cuentas prefabricadas y saldo de prueba. Se utiliza para validar integraciones sin coste de gas.

2.3. Volúmenes persistentes

Los datos que deben sobrevivir a la reconstrucción de contenedores se almacenan en volúmenes Docker gestionados.

Volumen	Contenido	Notas de backup
<code>postgres_data</code>	Ficheros de base de datos (WAL, tablas, índices)	Prioritario; usar <code>pg_dump</code> diariamente
<code>ipfs_data</code>	Repo del nodo Kubo (bloques, config, keystore)	Copiar volumen completo o exportar lista de pins
<code>ipfs_staging</code>	Buffer de importación temporal de IPFS	Puede eliminarse con seguridad
<code>certs</code>	Certificados SSL (Let's Encrypt)	Respaldar antes de renovaciones
<code>letsencrypt</code>	Estado y configuración de Certbot	Necesario para renovaciones automáticas

Cuadro 4: Volúmenes persistentes definidos en la infraestructura

2.4. Red interna Docker

Todos los servicios se conectan a una red de tipo `bridge` denominada `documentchain_network`. Esta segmentación permite:

- Resolución DNS por nombre de contenedor (p. ej., `backend` resuelve la IP interna del servicio homónimo).
- Aislamiento del tráfico interno respecto a la interfaz pública del anfitrión.
- Aplicación de políticas de cortafuegos por interfaz de red sin afectar al resto del sistema.

2.5. Flujo de peticiones

El recorrido típico de una solicitud de usuario se describe a continuación:

1. El usuario introduce `https://documentchain.ejemplo.com` en su navegador.
2. La petición DNS resuelve la dirección IP pública del VPS y llega al puerto 443.
3. Nginx recibe la conexión, valida el certificado SSL y aplica cabeceras de seguridad.
4. Si la URI comienza por `/api`, Nginx reenvía la petición al contenedor `backend:3000`.
5. Si la URI corresponde a un recurso estático o a una ruta de la SPA, Nginx sirve los ficheros del contenedor `frontend`.
6. El backend procesa la lógica de negocio y, en caso necesario, consulta:
 - PostgreSQL para datos relacionales a través de la capa de acceso a base de datos.
 - IPFS para almacenar o recuperar bloques de documentos cifrados.
 - El nodo blockchain (Hardhat local o una red de pruebas) para emitir o consultar transacciones.
7. Las respuestas viajan por el camino inverso hasta el navegador del usuario.

3. Instalacion paso a paso

3.1. Paso 1: Clonar el repositorio

Se recomienda clonar el repositorio oficial en un directorio de trabajo permanente, por ejemplo `/opt/documentchain`.

Listing 1: Clonación del código fuente

```
# Crear directorio de trabajo
sudo mkdir -p /opt/documentchain
sudo chown $USER:$USER /opt/documentchain
cd /opt/documentchain

# Clonar repositorio
git clone https://github.com/usuario/documentchain.git .
git checkout main
```

Tras la clonación, la estructura de directorios principal debe contener al menos las carpetas `frontend/`, `backend/`, `smart-contracts/`, `nginx/` y el archivo `docker-compose.yml`.

3.2. Paso 2: Configurar variables de entorno

El archivo `.env` centraliza toda la parametrización sensible y de infraestructura. Se proporciona una plantilla de ejemplo que debe copiarse y editarse.

Listing 2: Preparación del archivo de entorno

```
cp .env.example .env
chmod 600 .env
nano .env
```

La tabla 3.2 detalla las variables más críticas del sistema.

Cuadro 5: Variables de entorno principales de DocumentChain

Variable	Descripción	Ejemplo	Obl.
DATABASE_URL	Cadena de conexión a PostgreSQL con credenciales y nombre de base de datos	<code>postgresql://user:pass@postgres:5432</code>	Sí
JWT_SECRET	Clave simétrica para firma de tokens de acceso (mínimo 256 bits)	Generar con <code>openssl rand -hex 32</code>	Sí
JWT_EXPIRES_IN	Tiempo de validez del token de acceso (formato vercel/ms)	15m	Sí
REFRESH_SECRET	Clave independiente para tokens de refresco	Generar con <code>openssl rand -hex 32</code>	Sí
ENCRYPTION_KEY	Clave maestra para cifrado de documentos (AES-256-GCM)	Generar con <code>openssl rand -hex 32</code>	Sí
IPFS_API_URL	Endpoint interno de la API de IPFS	<code>http://ipfs:5001</code>	Sí

Continúa en la página siguiente

Cuadro 5 – continuación

Variable	Descripción	Ejemplo	Obl.
IPFS_GATEWAY_URL	URL pública o interna del gateway IPFS	http://ipfs:8080	No
BLOCKCHAIN_RPC_URL	URL del proveedor JSON-RPC (Hardhat local o nodo remoto)	http://hardhat:8545	Sí
CONTRACT_ADDRESS	Dirección desplegada del contrato maestro	0x1234...abcd	Sí
ADMIN_ROLE	Identificador hash del rol administrador en el contrato	0x...	Sí
SMTP_HOST	Servidor SMTP al que se conecta Nodemailer	smtp-relay.brevo.com	Sí
SMTP_PORT	Puerto del servicio SMTP	587 o 25	Sí
SMTP_SECURE	Indica si se utiliza TLS implícito (true para 465)	false	No
SMTP_USER	Usuario de autenticación SMTP (proporcionado por Brevo)	usuario@ejemplo.com	Sí
SMTP_PASS	Contraseña de autenticación SMTP	supersecreto	No
EMAIL_FROM	Dirección de remitente visible en los correos enviados	noreply@ejemplo.com	Sí
EMAIL_FROM_NAME	Nombre descriptivo del remitente	DocumentChain	No
FRONTEND_URL	URL base para enlaces de verificación y restablecimiento	https://ejemplo.com	Sí
una red de pruebas_RPC_URL	Endpoint de nodo en testnet una red de pruebas (Alchemy/Infura)	https://eth-una red de pruebas.g.alchemy.com/v2/...	No
PRIVATE_KEY	Clave privada de la cuenta desplegada en testnet/mainnet	0xabc...	No
ETHERSCAN_API_KEY	Clave de API para verificación de código fuente en Etherscan	ABC123...	No

3.3. Paso 3: Levantar servicios con Docker Compose

Una vez configurado el entorno, se procede a la construcción e inicio de los servicios. Se recomienda ejecutar los comandos en el orden indicado para facilitar la depuración de posibles errores.

Listing 3: Construcción e inicio de la plataforma

```
# Descargar imágenes base más recientes
docker compose pull
```

```
# Construir im\{a}genes locales sin cach\{e} de capas obsoletas
docker compose build --no-cache

# Iniciar todos los servicios en segundo plano
docker compose up -d

# Verificar estado de los contenedores
docker compose ps

# Inspeccionar logs del backend (Ctrl+C para salir)
docker compose logs -f --tail=100 backend
```

Tras ejecutar `docker compose up -d`, conviene esperar entre 30 y 60 segundos para que PostgreSQL complete su inicialización y el backend termine de compilar sus dependencias. El estado `healthy` o `Up` en la columna `STATUS` de `docker compose ps` indica que el servicio ha arrancado correctamente.

3.4. Paso 4: Configurar IPFS

El nodo IPFS requiere una inicialización previa y la habilitación de cabeceras CORS para permitir las peticiones desde el backend.

Listing 4: Configuración del nodo IPFS

```
# Inicializar el repositorio con perfil de servidor (sin anuncio en DHT local innecesario)
docker compose exec ipfs ipfs init --profile=server

# Habilitar CORS en la API
docker compose exec ipfs ipfs config --json API.HTTPHeaders.Access-Control-Allow-Origin '["*"]'
docker compose exec ipfs ipfs config --json API.HTTPHeaders.Access-Control-Allow-Methods '["PUT","POST","GET"]'
docker compose exec ipfs ipfs config --json API.HTTPHeaders.Access-Control-Allow-Headers '["Authorization","Content-Type"]'

# Reiniciar el contenedor para aplicar cambios
docker compose restart ipfs

# Verificar identidad y conectividad
docker compose exec ipfs ipfs id
```

3.4.1. Alternativa: proveedor gestionado IPFS (Pinata)

Ademas del nodo Kubo autoalojado, el backend puede operar con un proveedor gestionado como **Pinata** cuando se requiera simplificar la operacion o disponer de una capa externa de persistencia. Esta modalidad mantiene el mismo flujo funcional del sistema (generacion de CID, registro en blockchain y trazabilidad), sustituyendo unicamente el backend de almacenamiento.

Configuracion de Pinata en backend:

Listing 5: Configuración de proveedor IPFS Pinata

```
# backend/.env
IPFS_PROVIDER=pinata
PINATA_JWT=tu_jwt_token
# opcionales
PINATA_API_KEY=tu_api_key
PINATA_API_SECRET=tu_api_secret
PINATA_GATEWAY_URL=https://tu-gateway.mypinata.cloud
```

Notas operativas:

- Si `IPFS_PROVIDER=pinata`, no se requiere exponer `IPFS_API_URL` para subida y pinning en ese entorno.

- Si `IPFS_PROVIDER=self-hosted`, se utilizan `IPFS_API_URL` e `IPFS_GATEWAY_URL` del nodo Kubo.
- Ambas modalidades pueden coexistir por entorno (por ejemplo, desarrollo con Kubo y despliegue concreto con Pinata).

3.4.2. Configuración del nodo propio como nodo de red

Si se opta por mantener un nodo Kubo autoalojado y se desea que participe activamente en la red IPFS (compartiendo contenido con otros nodos y contribuyendo a la descentralización), es necesario realizar una configuración adicional para abrir el nodo a la red pública:

Listing 6: Apertura del nodo IPFS a la red pública

```
# Habilitar el perfil de servidor para comportamiento como nodo de red
docker compose exec ipfs ipfs config profile apply server

# Configurar el puerto del swarm para que sea accesible desde el exterior
# (requiere abrir el puerto 4001 en el firewall/ router)
docker compose exec ipfs ipfs config Addresses.Swarm --json \
'["ip4/0.0.0.0/tcp/4001","ip6:::tcp/4001"]'

# Habilitar el gateway p\ublico si se desea servir contenido via HTTP
docker compose exec ipfs ipfs config Addresses.Gateway /ip4/0.0.0.0/tcp/8080

# Configurar el anuncio de direcciones externas (reemplazar con IP/dominio real)
docker compose exec ipfs ipfs config --json Addresses.AppendAnnounce \
'["dns4/tu-dominio.com/tcp/4001","dns4/tu-dominio.com/tcp/4001/wss"]'

# Ajustar l'imites de almacenamiento (GC) para producci'on
docker compose exec ipfs ipfs config Datastore.StorageMax 500GB
docker compose exec ipfs ipfs config --json Datastore.StorageGCWatermark 90
docker compose exec ipfs ipfs config --json Swarm.ConnMgr.HighWater 500
docker compose exec ipfs ipfs config --json Swarm.ConnMgr.LowWater 250

# Reiniciar para aplicar cambios
docker compose restart ipfs
```

Consideraciones de seguridad para el nodo abierto:

- Restringir la API administrativa (puerto 5001) a `127.0.0.1` para evitar accesos no autorizados.
- Configurar el firewall para permitir solo los puertos necesarios: 4001 (swarm TCP), 8080 (gateway opcional).
- Monitorizar el uso de ancho de banda, por lo que el nodo retransmitirá bloques solicitados por otros pares de la red.
- Establecer límites de almacenamiento (`StorageMax`) para evitar el agotamiento del disco.
- Considerar el uso de una VPN o red privada si el contenido gestionado es sensible, limitando la comunicación del swarm a nodos autorizados.

En la configuración por defecto de DocumentChain, el proveedor IPFS depende de `IPFS_PROVIDER`. Con `self-hosted`, el nodo Kubo puede operar en modo local (sin anunciarse a la red pública) por simplicidad de despliegue y persistencia en volumen Docker. Con `pinata`, la persistencia se delega en el servicio gestionado manteniendo el mismo contrato funcional de CIDs.

3.5. Paso 5: Desplegar smart contracts

El despliegue varía según el entorno objetivo: red local de desarrollo (Hardhat) o testnet pública (una red de pruebas).

3.5.1. Despliegue en red local

Listing 7: Despliegue local con Hardhat

```
cd smart-contracts
npm ci

# En una terminal aparte, si no se usa el contenedor hardhat:
npx hardhat node

# En la terminal principal:
npx hardhat run scripts/deploy.ts --network localhost

# Copiar la direcci'\{o}n impresa en consola al archivo .env del backend
echo "CONTRACT_ADDRESS=0x..." >> ../backend/.env
```

3.5.2. Despliegue en una red de pruebas testnet

Listing 8: Despliegue en una red de pruebas

```
export una red de pruebas_RPC_URL=https://eth-una red de pruebas.g.alchemy.com/v2/TU_API_KEY
export PRIVATE_KEY=0xTU_CLAVE_PRIVADA_SIN_PREFIJO_0_CON_EL

npx hardhat run scripts/deploy.ts --network una red de pruebas
```

Es imprescindible contar con saldo de ETH de prueba en la cuenta desplegada. Los faucets públicos de una red de pruebas permiten solicitar fondos gratuitos introduciendo la dirección de la wallet. La dirección resultante del contrato debe registrarse en el archivo `.env` antes de reiniciar el backend.

3.6. Paso 6: Seed de base de datos

Una vez que PostgreSQL y el backend están operativos, se aplican las migraciones y se ejecuta el sembrado inicial.

Listing 9: Migraciones y seed de la base de datos

```
# Aplicar migraciones pendientes
docker compose exec backend npx prisma migrate deploy

# Ejecutar script de seed (usuarios de prueba, configuraci'\{o}n base)
docker compose exec backend npx prisma db seed

# Verificar tablas creadas
docker compose exec postgres psql -U postgres -d documentchain -c "\dt"
```

3.7. Paso 7: Configurar Nginx

El proxy inverso centraliza el acceso público y aplica políticas de seguridad. A continuación se presenta una configuración de producción completa.

Listing 10: Archivo de configuración de Nginx

```
# /opt/documentchain/nginx/nginx.conf
server {
    listen 80;
    listen [::]:80;
    server_name documentchain.ejemplo.com;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
```

```
listen [::]:443 ssl http2;
server_name documentchain.ejemplo.com;

ssl_certificate /etc/letsencrypt/live/documentchain.ejemplo.com/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/documentchain.ejemplo.com/privkey.pem;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers 'ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';
ssl_prefer_server_ciphers off;

add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;

gzip on;
gzip_vary on;
gzip_proxied any;
gzip_types text/plain text/css application/json application/javascript text/xml;

location / {
    root /usr/share/nginx/html;
    index index.html;
    try_files $uri $uri/ /index.html;
}

location /api {
    proxy_pass http://backend:3000;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 90s;
}

location /ipfs {
    proxy_pass http://ipfs:8080;
    proxy_set_header Host $host;
    proxy_read_timeout 300s;
}
}
```

Tras modificar la configuración, debe reiniciarse el contenedor Nginx:

```
docker compose restart nginx
```

3.8. Verificación post-instalación

Una vez finalizados los pasos anteriores, debe completarse la siguiente lista de comprobación:

1. Todos los contenedores aparecen en estado Up o healthy tras ejecutar `docker compose ps`.
2. PostgreSQL responde a consultas internas: `docker compose exec postgres pg_isready -U postgres`.
3. El endpoint de salud del backend devuelve estado 200: `curl -f http://localhost:3000/health`.
4. El frontend carga sin errores 500: `curl -f http://localhost/`.
5. La conexión HTTPS es válida: `curl -I https://documentchain.ejemplo.com`.
6. La API de IPFS responde: `curl http://localhost:5001/api/v0/version`.
7. El contrato inteligente está desplegado y su dirección figura en el archivo `.env`.
8. La base de datos contiene registros de semilla: consultar la tabla `User` desde PostgreSQL.

9. Se envía correctamente un correo de prueba desde el formulario de registro o mediante los logs del backend.
10. La wallet de prueba conecta a la red configurada (Hardhat o una red de pruebas) y puede firmar transacciones.

4. Configuración completa del correo electrónico

El subsistema de correo utiliza Brevo como proveedor SMTP externo. El backend genera los mensajes mediante Nodemailer y los entrega directamente a los servidores SMTP de Brevo, que se encargan de la entrega final a los destinatarios. Esta arquitectura evita que la lógica de negocio dependa de un proveedor concreto y permite sustituir el servicio de correo sin reescribir la capa de notificaciones del backend.

La configuración se centraliza en las variables de entorno `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER` y `SMTP_PASS`, que contienen las credenciales proporcionadas por Brevo. No se requiere ningún contenedor adicional de agente de transferencia ni configuración de relay.

4.1. Diagrama de flujo de envío de correos

La figura 2 representa el recorrido completo de un mensaje desde la acción del usuario hasta la bandeja del destinatario.

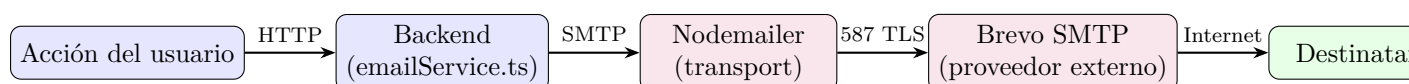


Figura 2: Flujo de envío de correos electrónicos en DocumentChain

4.2. Proceso interno de envío

El proceso de envío comienza en el servicio `emailService.ts`. Este servicio compone el asunto, selecciona la plantilla HTML adecuada y resuelve las URLs visibles en el correo a partir de `FRONTEND_URL`. Cuando se dispara una acción como registro, verificación de correo, recuperación de contraseña o aviso de documento compartido, el backend renderiza la plantilla con *Handlebars* y delega el transporte en Nodemailer.

Nodemailer se conecta directamente al servicio SMTP de Brevo utilizando las credenciales configuradas en el archivo `.env`. Brevo negocia TLS, autentica el remitente y entrega el mensaje al destinatario final. El flujo puede describirse como *backend* → *Brevo SMTP* → *destinatario*.

4.3. Configuración SMTP en el backend

La comunicación con Brevo se configura exclusivamente a través de las variables de entorno del backend. No se requiere ningún contenedor adicional de relay ni archivos de configuración del sistema de correo.

Listing 11: Variables SMTP en el archivo `.env`

```

SMTP_HOST=smtp-relay.brevo.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=usuario@ejemplo.com
SMTP_PASS=clave-api-brevo
EMAIL_FROM=noreply@documentchain.ejemplo.com
EMAIL_FROM_NAME=DocumentChain
  
```

Las credenciales `SMTP_USER` y `SMTP_PASS` se obtienen del panel de control de Brevo, en la sección *SMTP & API*. Es necesario que el dominio o la dirección de correo utilizada como remitente esté verificada en dicho panel para que los mensajes se entreguen correctamente.

4.4. Configuración de Nodemailer

El backend inicializa el transporte SMTP mediante el siguiente fragmento de código TypeScript.

Listing 12: Configuración del transporte Nodemailer

```
import nodemailer from 'nodemailer';

const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST || 'smtp-relay.brevo.com',
  port: Number(process.env.SMTP_PORT) || 587,
  secure: process.env.SMTP_SECURE === 'true',
  auth: {
    user: process.env.SMTP_USER || undefined,
    pass: process.env.SMTP_PASS || undefined,
  },
  tls: {
    rejectUnauthorized: false, // En entornos Docker internos con certs autofirmados
  },
});

export async function sendEmail(to: string, subject: string, html: string) {
  await transporter.sendMail({
    from: `${process.env.EMAIL_FROM_NAME} <${process.env.EMAIL_FROM}>`,
    to,
    subject,
    html,
  });
}
```

4.5. Variables relevantes de correo

La tabla 6 recoge todas las variables de entorno vinculadas al subsistema de correo.

Variable	Función
<code>SMTP_HOST</code>	Servidor SMTP de Brevo al que se conecta el backend.
<code>SMTP_PORT</code>	Puerto del SMTP utilizado por el backend.
<code>SMTP_SECURE</code>	Indica si el transporte backend hacia SMTP emplea TLS implícito.
<code>SMTP_USER</code> , <code>SMTP_PASS</code>	Credenciales de autenticación si el backend entrega a un SMTP externo o autenticado.
<code>EMAIL_FROM</code> , <code>EMAIL_FROM_NAME</code>	Remitente lógico mostrado al usuario.
<code>FRONTEND_URL</code>	URL base empleada para enlaces de verificación, establecimiento y navegación desde correos.

Cuadro 6: Variables de entorno relacionadas con el subsistema de correo

4.6. Tipos de correo y criterios de envío

Tipo de correo	Evento disparador	Destinatario	Plantilla
Verificación de cuenta	Finalización del registro	Usuario registrado	verify-email.hbs
Restablecimiento de contraseña	Solicitud desde login	Usuario solicitante	reset-password.hbs
Confirmación de cambio	Actualización exitosa	Usuario afectado	password-changed.hbs
Bienvenida	Verificación de email completada	Usuario verificado	welcome.hbs
Documento compartido	Acción de compartir	Colaborador invitado	shared-document.hbs
Notificación operativa	Evento del sistema (fallo, alerta)	Administrador	notification.hbs

Cuadro 7: Tipos de correo activos en la plataforma

4.7. Resolución de problemas de entrega

Cuando los correos no llegan a su destino, conviene revisar los siguientes aspectos:

- **Credenciales SMTP:** verificar que las variables `SMTP_USER` y `SMTP_PASS` en el archivo `.env` coinciden con las credenciales proporcionadas por Brevo.
- **Remitente verificado:** en Brevo es obligatorio verificar el dominio o la dirección de correo desde la que se envía. Comprobar que `EMAIL_FROM` corresponde a un remitente autorizado en el panel de Brevo.
- **Límites de envío:** los planes de Brevo imponen límites diarios y por hora. Si se superan, los mensajes se encolan y pueden retrasarse.
- **Puerto SMTP:** algunos proveedores de VPS bloquean el puerto 25 de salida. Asegurarse de usar el puerto 587 en `SMTP_PORT`.

5. Configuración de blockchain

5.1. Despliegue en red local (Hardhat network)

Para el desarrollo y las pruebas de integración se utiliza la red local integrada en Hardhat. Esta red simula un entorno Ethereum completo con bloques instantáneos y cuentas prefinanciadas.

Listing 13: Inicio de la red local y despliegue

```
cd smart-contracts
npx hardhat node
# La red escucha en http://127.0.0.1:8545 y expone 20 cuentas con 10000 ETH cada una

# En otra terminal, dentro del mismo directorio:
npx hardhat run scripts/deploy.ts --network localhost
```

La dirección impresa en consola debe asignarse a la variable `CONTRACT_ADDRESS` del backend. En modo Docker, el contenedor `hardhat` ejecuta automáticamente el nodo si la imagen está configurada para ello; de lo contrario, puede mapearse el puerto 8545 del anfitrión al contenedor backend.

5.2. Despliegue en una red de pruebas testnet

una red de pruebas es la testnet pública recomendada para validar contratos antes de un despliegue en mainnet.

1. **Faucet:** obtener ETH de prueba desde un faucet público (p. ej., una red de pruebas faucet.com, Alchemy faucet) enviando fondos a la dirección de la wallet desplegada.
2. **Configuración:** añadir la red una red de pruebas al archivo `hardhat.config.ts`.
3. **Despliegue:** ejecutar el script de despliegue apuntando a la red configurada.
4. **Verificación:** publicar el código fuente en Etherscan para facilitar la auditoría pública.

Listing 14: Fragmento de configuración de red una red de pruebas en Hardhat

```
// hardhat.config.ts
import { HardhatUserConfig } from 'hardhat/config';

const config: HardhatUserConfig = {
  networks: {
    una red de pruebas: {
      url: process.env.una red de pruebas_RPC_URL || '',
      accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : [],
    },
  },
  etherscan: {
    apiKey: process.env.ETHERSCAN_API_KEY || '',
  },
};

export default config;
```

5.3. Configuración de variables CONTRACT_ADDRESS y ADMIN_ROLE

Tras el despliegue, el script `deploy.ts` emite en consola dos valores esenciales:

- **CONTRACT_ADDRESS:** dirección del contrato desplegado en la red activa.
- **ADMIN_ROLE:** hash keccak256 del rol administrador (típicamente `keccak256('ADMIN_ROLE')`).

Ambos valores deben trasladarse al archivo `.env` del backend y del frontend (si el frontend lee la dirección del contrato de forma pública). Un reinicio de los contenedores es necesario para que las nuevas variables se carguen en memoria.

5.4. Verificación del contrato en Etherscan

La verificación permite a terceros inspeccionar el código fuente asociado a una dirección on-chain.

Listing 15: Verificación en Etherscan desde Hardhat

```
npx hardhat verify --network una red de pruebas CONTRACT_ADDRESS "Argumento1" "Argumento2"
```

El comando requiere que la variable `ETHERSCAN_API_KEY` esté exportada en el entorno. Tras la ejecución exitosa, la página del contrato en Etherscan mostrará el código fuente Solidity y la interfaz de lectura/escritura pública.

5.5. Configuración del event listener

El backend sincroniza ciertos estados escuchando eventos emitidos por el contrato. Esta sincronización se realiza mediante `ethers.js` y un loop de polling o suscripción a WebSocket si el proveedor RPC lo soporta.

Listing 16: Esqueleto del listener de eventos

```
import { ethers } from 'ethers';
import { contractABI } from './abi';

const provider = new ethers.JsonRpcProvider(process.env.BLOCKCHAIN_RPC_URL);
const contract = new ethers.Contract(
  process.env.CONTRACT_ADDRESS!,
  contractABI,
  provider
);

contract.on('DocumentCreated', async (docId, owner, timestamp, event) => {
  // Lógica de sincronización: actualizar estado en PostgreSQL
  console.log(`Documento ${docId} registrado por ${owner} en ${timestamp}`);
});
```

En producción se recomienda envolver el listener en un proceso independiente o en un servicio cron que recupere eventos históricos mediante `contract.queryFilter`, garantizando así la tolerancia a reinicios del backend.

6. Uso de Docker y Nginx

En este apartado se detalla la configuración de Docker como mecanismo de reproducibilidad del entorno y de Nginx como proxy inverso del sistema.

Docker se ha elegido como mecanismo principal de reproducibilidad del entorno. Al encapsular PostgreSQL, Hardhat, backend y frontend en contenedores, se elimina casi por completo la necesidad de que una tercera persona instale dependencias manualmente. Con un `docker compose up -d` basta para que todo arranque.

En cuanto a Nginx, cumple una función concreta dentro del frontend dockerizado: sirve los archivos estáticos generados por Vite, resuelve el enrutado de la SPA correctamente y redirige las peticiones a `/api` hacia el backend. Durante las pruebas de despliegue se observó que, sin esta capa, era necesario improvisar soluciones para exponer el frontend o añadir lógica adicional en otros servicios.

6.1. Comandos Docker de uso frecuente

Comando	Propósito
<code>docker compose ps</code>	Listar contenedores activos y su estado de salud
<code>docker compose logs -f --tail=200 servicio</code>	Seguir en tiempo real los últimos 200 logs de un servicio
<code>docker compose restart servicio</code>	Reiniciar un servicio específico sin afectar al resto
<code>docker compose build --no-cache servicio</code>	Reconstruir la imagen descartando cachés previos
<code>docker compose exec servicio sh</code>	Abrir un shell interactivo dentro del contenedor
<code>docker system prune -af --volumes</code>	Eliminar imágenes, contenedores detenidos y volúmenes no utilizados (uso con precaución)
<code>docker volume ls</code>	Listar volúmenes persistentes
<code>docker stats</code>	Mostrar consumo de CPU y memoria en tiempo real

Cuadro 8: Comandos Docker frecuentes en operativa diaria

6.2. Escalado y healthchecks en Docker Compose

Aunque el diseño actual no requiere múltiples réplicas del backend, Docker Compose permite escalar horizontalmente un servicio de la siguiente manera:

```
docker compose up -d --scale backend=2
```

Para que el escalado sea efectivo, el proxy inverso debería distribuir la carga entre las instancias mediante un balanceador upstream.

Los **healthchecks** definen condiciones para considerar un contenedor saludable. El siguiente fragmento ilustra su inclusión en el servicio backend:

Listing 17: Ejemplo de healthcheck en docker-compose.yml

```
services
  backend
    build./backend
    healthcheck
      test["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval30s
      timeout10s
      retries3
      start_period40s
```

6.3. Configuración avanzada de Nginx

Además del proxy inverso, Nginx puede aplicar políticas de seguridad y rendimiento.

Listing 18: Directivas adicionales de seguridad y rendimiento en Nginx

```
# Rate limiting: mitigar abuso de endpoints de login
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;

server {
    # ... configuracion base ...

    location /api/auth/login {
        limit_req zone=login burst=3 nodelay;
        proxy_pass http://backend:3000;
    }

    # Compression gzip
```

```
gzip on;
gzip_vary on;
gzip_proxied any;
gzip_types text/plain text/css application/json application/javascript text/xml application/xml;

# Cabeceras de seguridad
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none';" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
}
```

6.4. Certificados SSL auto-renovables

La combinación de Certbot con un hook de recarga garantiza que los certificados se renueven automáticamente sin intervención manual.

Listing 19: Renovación automática con recarga de Nginx

```
# Script de renovacion desplegado en /etc/cron.d/certbot-documentchain
0 3 * * * root certbot renew --quiet --deploy-hook \
    "docker compose -f /opt/documentchain/docker-compose.yml exec nginx nginx -s reload"
```

7. GitHub Actions y despliegue continuo

En este apartado se describe el pipeline de integracion y despliegue continuo configurado en el repositorio.

El repositorio incorpora un *workflow* de GitHub Actions orientado a *runners* autoalojados en Linux. Cuando llega un *push* a la rama configurada, ese *runner* actualiza el código, reconstruye las imágenes Docker necesarias, aplica migraciones Prisma con la imagen del backend y relanza el conjunto mediante Docker Compose.

7.1. Workflow completo de CI/CD

El archivo `.github/workflows/deploy.yml` define el pipeline completo.

Listing 20: Pipeline de despliegue continuo (deploy.yml)

```
name: CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run linter
```

```
runnpm run lint

- nameRun unit tests
  runnpm run test

- nameBuild frontend
  runnpm run buildfrontend

- nameBuild backend
  runnpm run buildbackend

deploy
needsbuild-and-test
ifgithub.ref == 'refs/heads/main'
runs-onubuntu-latest
steps
- nameDeploy to VPS via SSH
  usesappleboy/ssh-action@v1.0.3
  with
    host${{ secrets.VPS_HOST }}
    username${{ secrets.VPS_USER }}
    key${{ secrets.SSH_PRIVATE_KEY }}
  script|
    set -e
    cd /opt/documentchain
    git pull origin main
    echo "${{ secrets.ENV_FILE }}" > .env
    docker compose -f docker-compose.yml pull
    docker compose -f docker-compose.yml build --no-cache
    docker compose -f docker-compose.yml up -d
    docker compose exec backend npx prisma migrate deploy
    docker compose ps
```

7.2. Descripción del pipeline

El pipeline se divide en dos fases diferenciadas:

1. **Construcción y pruebas:** se ejecuta en un entorno limpio de Ubuntu proporcionado por GitHub. Incluye la instalación de dependencias, el análisis estático de código (linting), la ejecución de tests unitarios y la compilación de los artefactos de frontend y backend. Si alguno de estos pasos falla, el despliegue se cancela automáticamente.
2. **Despliegue:** depende del éxito de la fase anterior y sólo se activa para la rama `main`. Mediante una conexión SSH se accede al VPS, se actualiza el código fuente, se reconstruyen las imágenes Docker y se aplican las migraciones de base de datos.

7.3. Configuración de secrets en GitHub

Desde la interfaz de GitHub, en *Settings > Secrets and variables > Actions*, deben configurarse las siguientes claves:

Secret	Contenido
VPS_HOST	Dirección IP o nombre de dominio del servidor de producción
VPS_USER	Nombre de usuario con permisos de despliegue (p. ej., <code>deployer</code>)
SSH_PRIVATE_KEY	Clave privada SSH en formato PEM; la pública debe estar en <code>~/.ssh/authorized_keys</code> del VPS
ENV_FILE	Contenido completo del archivo <code>.env</code> de producción (líneas separadas por <code>\n</code>)

Cuadro 9: Secrets requeridos en GitHub Actions

7.4. Estrategia de rollback

Si un despliegue introduce una regresión crítica, es posible revertir el sistema a un estado estable mediante alguna de las siguientes estrategias:

- **Rollback por Git:** ejecutar `git revert HEAD` o `git checkout <tag_estable>` en el servidor y repetir el ciclo de construcción.
- **Rollback por imagen Docker:** si se etiquetaron las imágenes anteriores con un tag de versión, basta con modificar el archivo `docker-compose.yml` para apuntar a la imagen previa y ejecutar `docker compose up -d`.
- **Rollback de base de datos:** restaurar el último volcado SQL válido mediante `psql` antes de reintentar el despliegue.

Se recomienda realizar backups de la base de datos inmediatamente antes de cada despliegue automatizado para minimizar la ventana de pérdida de datos.

8. Mantenimiento y actualización

8.1. Backup de base de datos

La política de respaldo de PostgreSQL debe ejecutarse de forma automática mediante una tarea programada (cron).

Listing 21: Script de backup de PostgreSQL

```
#!/bin/bash
# /opt/documentchain/scripts/backup-db.sh
BACKUP_DIR="/opt/backups/documentchain"
mkdir -p "$BACKUP_DIR"
FILENAME="db_$(date +%Y%m%d_%H%M%S).sql"

docker compose -f /opt/documentchain/docker-compose.yml exec -T postgres \
  pg_dump -U postgres -d documentchain --clean --if-exists > "$BACKUP_DIR/$FILENAME"

gzip "$BACKUP_DIR/$FILENAME"
find "$BACKUP_DIR" -name "db_*.sql.gz" -mtime +7 -delete
```

La entrada cron correspondiente:

```
0 2 * * * /opt/documentchain/scripts/backup-db.sh >> /var/log/db-backup.log 2>&1
```

8.2. Backup de IPFS

El nodo IPFS almacena bloques de contenido que deben preservarse. Además de copiar el volumen `ipfs_data`, se recomienda exportar la lista de objetos fijados (pins).

```
docker compose exec ipfs ipfs pin ls --type=recursive > /opt/backups/ipfs-pins-$(date +%Y%m%d).txt
docker run --rm -v documentchain_ipfs_data:/data -v /opt/backups:/backup alpine tar czf /backup/ipfs_data_$(date +%Y%m%d).tar.gz -C /data .
```

8.3. Actualizacion segura del sistema

El siguiente procedimiento minimiza el riesgo de pérdida de datos durante una actualización:

1. Realizar backup completo de PostgreSQL e IPFS.
2. Copiar el archivo `.env` a una ruta externa al repositorio.
3. Ejecutar `git fetch origin` y revisar las notas de la release.
4. Aplicar la actualización: `git checkout vX.Y.Z` o `git pull origin main`.
5. Reconstruir imágenes: `docker compose build --no-cache`.
6. Levantar servicios: `docker compose up -d`.
7. Aplicar migraciones: `docker compose exec backend npx prisma migrate deploy`.
8. Verificar healthchecks y logs durante 10 minutos.

8.4. Actualizacion de dependencias

Conviene auditar las dependencias Node.js con cierta periodicidad. Durante el desarrollo del proyecto, el autor ha seguido esta rutina para detectar vulnerabilidades antes de que se acumularan:

```
# Auditoria de seguridad
npm audit

# Actualizacion conservadora respetando rangos semver
npm update

# Listado de paquetes con versiones nuevas disponibles
npm outdated
```

Las actualizaciones de dependencias críticas (Prisma, Express, ethers.js) deben probarse primero en el entorno de desarrollo antes de desplegarse en producción.

8.5. Monitoreo de logs

La observabilidad del sistema se basa en los logs estándar de Docker, accesibles mediante los siguientes patrones:

```
# Logs en tiempo real filtrando errores
docker compose logs -f --tail=500 backend | grep -i "error\|exception\|fatal"

# Logs del sistema Docker (requiere privilegios)
journalctl -u docker.service -f -n 200
```


8.6. Limpieza de recursos

Con el tiempo, el sistema acumula imágenes huérfanas, contenedores detenidos y cachés de construcción que consumen espacio en disco.

```
# Eliminar objetos no utilizados (imagenes, redes, volúmenes huérfanos)
docker system prune -af

# ATENCION: el siguiente comando borra volúmenes no referenciados.
# Verificar previamente que no se pierdan postgres_data ni ipfs_data.
docker volume prune -f
```

9. Scripts de utilidad

Además de los procedimientos manuales, el proyecto incluye scripts que automatizan tareas repetitivas.

9.1. Script de backup automatizado

Listing 22: backup.sh - Respaldo integral automatizado

```
#!/bin/bash
set -euo pipefail

PROJECT_DIR="/opt/documentchain"
BACKUP_DIR="/opt/backups/documentchain"
DATE=$(date +%Y%m%d_%H%M%S)
mkdir -p "$BACKUP_DIR"

# Backup de PostgreSQL
docker compose -f "$PROJECT_DIR/docker-compose.yml" exec -T postgres \
  pg_dump -U postgres -d documentchain --clean --if-exists \
  | gzip > "$BACKUP_DIR/db_$DATE.sql.gz"

# Backup de .env
cp "$PROJECT_DIR/.env" "$BACKUP_DIR/env_$DATE.bak"

# Backup de volumen IPFS
docker run --rm \
  -v documentchain_ipfs_data:/data \
  -v "$BACKUP_DIR":/backup \
  alpine tar czf /backup/ipfs_$DATE.tar.gz -C /data .

echo "[$(date)] Backup completado: $DATE" >> "$BACKUP_DIR/backup.log"
```

9.2. Script de verificación de salud

Listing 23: healthcheck.sh - Verificación de estado del sistema

```
#!/bin/bash
FAILED=0

check_http() {
  local url=$1
  local name=$2
  if ! curl -sf "$url" > /dev/null; then
    echo "FAIL: $name no responde en $url"
    FAILED=1
  else
    echo "OK: $name"
  fi
}

check_http "http://localhost:3000/health" "Backend API"
```

```
check_http "http://localhost/" "Frontend"
check_http "http://localhost:5001/api/v0/version" "IPFS API"

if ! docker compose exec postgres pg_isready -U postgres > /dev/null 2>&1; then
    echo "FAIL: PostgreSQL no responde"
    FAILED=1
else
    echo "OK: PostgreSQL"
fi

exit $FAILED
```

9.3. Script de migración de datos

Listing 24: migrate-and-seed.sh - Migracion y poblacion de datos

```
#!/bin/bash
set -e

cd /opt/documentchain

echo "Aplicando migraciones..."
docker compose exec backend npx prisma migrate deploy

echo "Ejecutando seed..."
docker compose exec backend npx prisma db seed

echo "Migracion completada."
```

9.4. Script de generacion de seeds

Listing 25: generate-seed-data.sh - Generacion de datos de prueba masivos

```
#!/bin/bash
# Requiere acceso al contenedor backend y a la CLI de Prisma
cd /opt/documentchain
docker compose exec backend npx ts-node prisma/seed-massive.ts
```

9.5. Reseed de entorno de desarrollo

Listing 26: reseed-dev.ps1 - Reseteo completo del entorno de desarrollo

```
# Detener contenedores y eliminar vol\{u}menes
docker-compose down -v

# Reconstruir e iniciar
docker-compose up -d --build

# Esperar a que PostgreSQL este disponible
sleep 10

# Aplicar migraciones y seed
docker exec documentchain-backend npx prisma migrate deploy
docker exec documentchain-backend npx prisma db seed
```

10. Resolucion de incidencias comunes

A continuación se recogen los problemas que han aparecido con más frecuencia durante las pruebas de despliegue del autor, junto con su diagnóstico y la solución aplicada en cada caso.

Cuadro 10: Matriz de resolución de incidencias

Problema	Causa probable	Diagnóstico	Solución
No se puede conectar a PostgreSQL	Credenciales erróneas, contenedor caído o red mal configurada	<code>docker compose logs postgres;</code> verificar <code>pg_isready</code>	Revisar <code>DATABASE_URL</code> , reiniciar contenedor, comprobar red interna
IPFS no responde en puerto 5001	Nodo no inicializado, CORS bloqueado o contenedor reiniciándose	<code>curl http://localhost:5001/api/v0/repo/status</code>	Ejecutar <code>ipfs init</code> , <code>api/v0/repo/status</code> , revisar logs
Error CORS al subir archivo	Cabeceras CORS no aplicadas en IPFS o backend	Inspeccionar respuesta preflight en navegador	Configurar <code>API.HTTPHeaders</code> en Kubo y headers en Express
Smart contract no desplegado	Cuenta sin saldo, red incorrecta o error de compilación	Revisar saldo en Hardhat/una red de pruebas, verificar <code>hardhat.config.ts</code>	Recargar faucet, corregir red, recompilar con <code>npm ci</code>
Transacciones blockchain fallan	<code>CONTRACT_ADDRESS</code> incorrecto, red desconectada o gas insuficiente	Verificar dirección en <code>.env</code> , comprobar estado del nodo	Actualizar <code>.env</code> , reiniciar nodo, estimar gas adecuadamente
Frontend no carga (404 en rutas)	Nginx no redirige rutas SPA al <code>index.html</code>	<code>curl -I http://localhost/ruta-interna</code>	Añadir <code>try_files \$uri \$uri/ /index.html</code> en Nginx
Emails no se envían	Credenciales de Brevo incorrectas, dominio no verificado o puerto bloqueado	<code>docker compose logs backend;</code> revisar logs de Nodemailer	Verificar <code>SMTP_USER</code> y <code>SMTP_PASS</code> en <code>.env</code> , comprobar verificación de dominio en Brevo, probar conectividad a <code>smtp-relay.brevo.com:587</code>
Certificado SSL caducado	Renovación fallida o cron job ausente	<code>openssl s_client -connect dominio:443</code>	Ejecutar <code>certbot renew</code> manualmente, revisar cron
Wallet no detecta red Hardhat	Cadena ID incorrecta o RPC no accesible desde el navegador	Inspeccionar consola de MetaMask, probar <code>curl</code> al 8545	Añadir red manual en MetaMask, mapear puerto del host
Error de memoria al compilar contratos	Node.js sin límite suficiente de heap	<code>FATAL ERROR: Reached heap limit</code>	Ejecutar con <code>NODE_OPTIONS=--max-old-space-size=4096</code>

Continúa en la página siguiente

Cuadro 10 – continuación

Problema	Causa probable	Diagnóstico	Solución
Contenedores se reinician continuamente	Healthcheck fallido, puerto ocupado o dependencia no resuelta	<code>docker compose ps</code> ; revisar código de salida	Corregir configuración, liberar puertos, revisar logs previos al reinicio
Pérdida de datos de IPFS	Volumen no persistente o eliminación accidental con <code>prune</code>	Verificar existencia del volumen <code>ipfs_data</code>	Restaurar backup del volumen, re-pin contenido crítico
Latencia alta en peticiones	Falta de comprensión, queries lentos en PostgreSQL o saturación de red	Monitorizar <code>docker stats</code> , analizar slow queries	Habilitar gzip, añadir índices, escalar CPU/RAM
Error de conexión WalletConnect	Project ID inválido o dominio no registrado en WalletConnect Cloud	Consola del navegador muestra 401/403 en WalletConnect	Crear proyecto válido en WalletConnect Cloud, añadir dominio permitido
Base de datos corrupta	Apagado brusco del servidor o disco lleno	PostgreSQL entra en modo recovery o no arranca	Restaurar último backup válido, ejecutar <code>pg_resetwal</code> si es necesario

Además de los escenarios anteriores, el autor recomienda mantener un pequeño registro de incidencias en el propio servidor. Basta con anotar la fecha, el síntoma y la solución aplicada. Esta práctica, aunque sencilla, ayuda a identificar patrones recurrentes y a planificar mejoras preventivas en la infraestructura.

11. Referencias y bibliografía

1. Docker Inc. *Docker Documentation*. <https://docs.docker.com/>
2. Docker Inc. *Docker Compose Documentation*. <https://docs.docker.com/compose/>
3. Nginx. *Nginx Documentation*. <https://nginx.org/en/docs/>
4. GitHub. *GitHub Actions Documentation*. <https://docs.github.com/en/actions>
5. Let's Encrypt. *Certbot Documentation*. <https://certbot.eff.org/docs/>
6. Postfix. *Postfix Documentation*. <https://www.postfix.org/documentation.html>
7. Prisma. *Prisma CLI Reference*. <https://www.prisma.io/docs/reference/api-reference/command-reference>
8. Hardhat. *Hardhat Documentation*. <https://hardhat.org/docs>
9. IPFS. *Kubo (go-ipfs) Documentation*. <https://docs.ipfs.tech/install/command-line/>
10. Pinata. *Pinata Documentation*. <https://docs.pinata.cloud/>